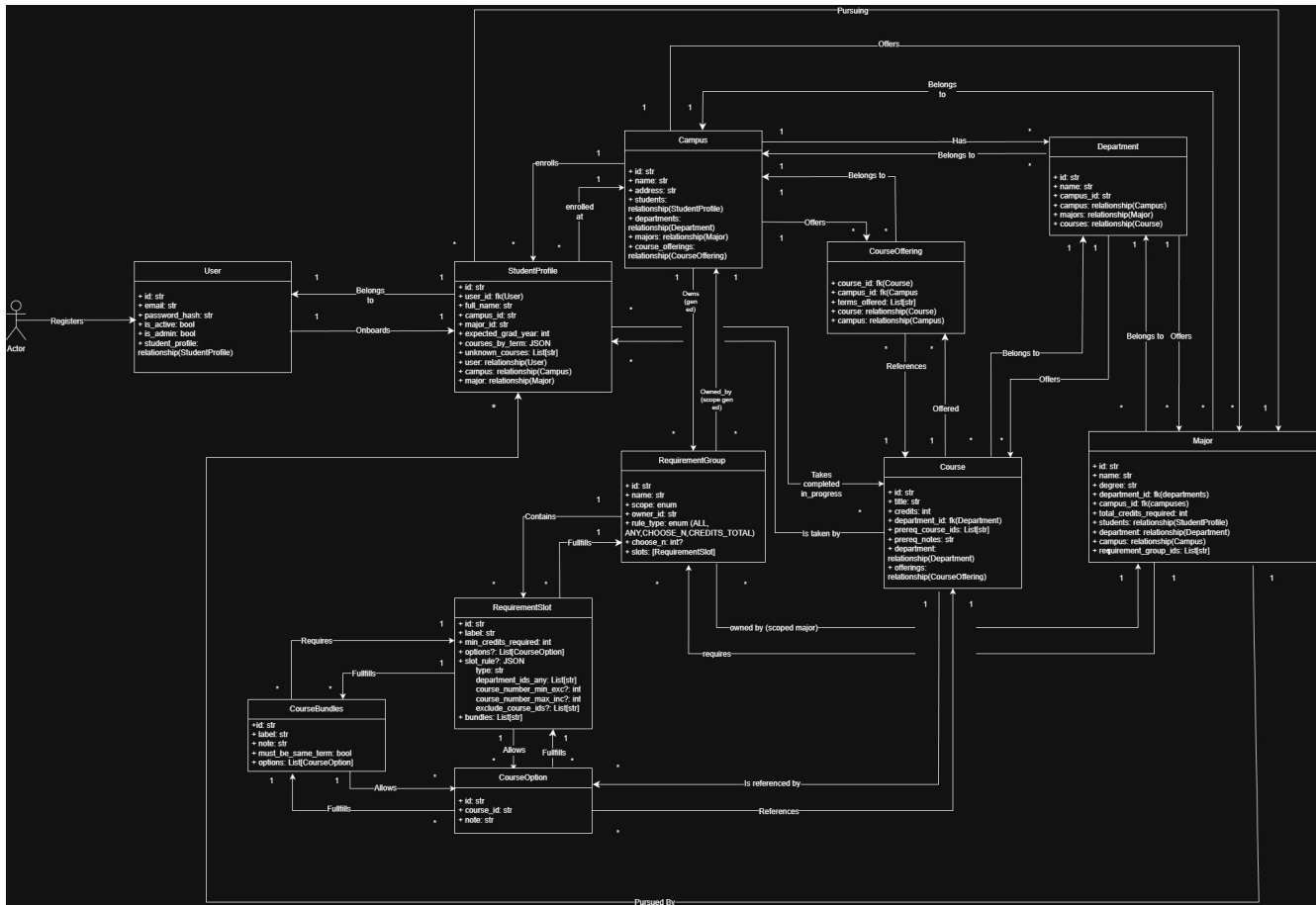


# UniBrain v1 UML Diagram - Plain-English Walkthrough

Reference guide for how the UniBrain data model fits together end-to-end.



This page shows the full UML for quick reference. The next pages explain it end-to-end in plain English.

# What you are looking at

This UML describes how UniBrain stores (1) a university catalog (campuses, departments, majors, courses, and which courses are offered where), and (2) a degree-requirements system (requirement groups, slots, and options/bundles) that UniBrain can use to audit a student's progress and recommend next steps.

## Two halves of the system

- Catalog side: Campus → Department → Major / Course, plus CourseOffering to say a course exists at a specific campus and which terms it's offered.
- Student side: User → StudentProfile (campus + major + the student's courses-by-term).
- Requirements side: Major references RequirementGroups. Each RequirementGroup contains RequirementSlots. Each slot is satisfied by either explicit course options, bundles of courses, or a bucket rule (credits-based filter).

# 1. People and identity

## User

A User is the login/account object (email + password hash, active/admin flags). Each user has exactly one StudentProfile (your app treats a user as a student once they complete onboarding).

## StudentProfile

StudentProfile is the student's academic identity inside UniBrain. It links the user to a chosen campus and major and stores the student's progress data.

- **campus\_id** and **major\_id**: which catalog context the student lives in.
- **expected\_grad\_year**: optional planning metadata.
- **courses\_by\_term** (JSON): the student's term-structured record (completed + in-progress, grouped by term). This enables constraints like "lecture + lab must be in the same term".
- **unknown\_courses**: a safety valve list for anything the student typed that doesn't match the catalog yet.

## 2. The catalog: where courses live

### Campus

Campus is the top-level container for a school location (e.g., LIU Brooklyn). It has departments, majors, course offerings, and students enrolled at that campus.

### Department

Department belongs to a campus and groups related majors and courses (e.g., Computer Science department). In practice, `department_id` is a clean way to filter course buckets like “any HIS/ART/PHI course”.

### Course

Course is the atomic unit students take. It includes title, credits, and `department_id`. It also stores prerequisite logic as a list of course ids plus human-readable prereq notes.

- **prereq\_course\_ids**: structured prerequisites used for eligibility checks and recommendations.
- **prereq\_notes**: free-text caveats (permissions, test scores, overrides, etc.). This keeps V1 flexible without modeling every edge case as code.

### CourseOffering

CourseOffering ties a Course to a Campus and optionally records terms offered. This answers questions like “is CS105 offered at Brooklyn and when?”. It’s also the bridge you can later use for scheduling recommendations.

### Major

Major represents a program of study (e.g., BS in Computer Science). It belongs to a department and a campus, tracks total credits required, and points to the requirement groups that define how to graduate.

## 3. Degree requirements: how UniBrain checks graduation

### RequirementGroup

A RequirementGroup is a named section of requirements, owned by either a major or a campus (scope). Think of it like a heading in a degree audit: “Core Requirements”, “Math & Science”, “Major Electives”, etc. The group’s **rule\_type** describes how its slots combine.

- **ALL**: every slot in the group must be satisfied.
- **ANY**: satisfying any one slot satisfies the group.
- **CHOOSE\_N**: satisfy N slots out of the group’s slots (choose\_n).
- **CREDITS\_TOTAL**: credits are aggregated across slots (optional pattern; often you can keep credits on slots instead).

### RequirementSlot

A RequirementSlot is one checkable requirement inside a group. It is the unit UniBrain evaluates against the student’s record.

- **label**: what shows up in the UI (e.g., “Science with Lab”).
- **min\_credits\_required**: for credits-based slots (e.g., “12 credits of liberal arts”).
- **slot\_rule** (JSON): rules for bucket-style slots, such as department filters and course-number bounds.
- **options**: explicit allowed courses (simple OR list).
- **bundles**: OR between bundles, AND inside a bundle.

### CourseOption

CourseOption is a pointer to a specific course\_id (with an optional note). It’s used when the catalog explicitly lists acceptable courses for a slot.

### CourseBundles

CourseBundles represent one complete ‘path’ to satisfy a slot. A bundle holds a list of CourseOptions that must all be satisfied together. The **must\_be\_same\_term** flag enforces constraints like “lecture and lab must be taken in the same term”.

## 4. End-to-end walkthrough: from signup to degree audit

### Step A - User registers

A person creates a User (email + password). At this point they are just an account.

### Step B - Onboarding creates StudentProfile

During onboarding, the user selects a campus and major, enters their expected graduation year, and inputs courses by term. UniBrain stores this in StudentProfile.courses\_by\_term and keeps any unrecognized course codes in unknown\_courses.

### Step C - UniBrain loads the catalog context

Given campus\_id and major\_id, UniBrain can pull: departments, courses, and course offerings for that campus, plus the requirement groups referenced by the major.

### Step D - UniBrain evaluates requirement groups and slots

- For each RequirementGroup attached to the major, UniBrain checks slots using the group's rule\_type (ALL/ANY/CHOOSE\_N/etc.).
- For each RequirementSlot, UniBrain chooses a satisfaction strategy based on what is present:
  - **Explicit options:** satisfied if the student completed any of the option course\_ids.
  - **Bundles:** satisfied if the student completed all courses in at least one bundle. If must\_be\_same\_term=true, those courses must appear together in the same term entry.
  - **Bucket rule:** satisfied if matching completed courses sum to at least min\_credits\_required credits.

### Step E - Prerequisites help recommendations (not history)

Prerequisites on Course are mainly used when recommending next courses. UniBrain should not silently add missing prereqs to the student's record. Instead it can avoid recommending prereqs for a course the student already completed and optionally flag record inconsistencies.

## 5. Concrete examples mapped to the model

### Example 1 - Simple OR slot (English Literature)

Slot label: "English Literature". Options: ENG101 or ENG102. The student satisfies the slot by completing either course.

### Example 2 - Bundle slot (Science with Lab, same term)

Slot label: "Science with Lab". Bundles: (BIO lecture + BIO lab) OR (CHEM lecture + CHEM lab) OR (PHY lecture + PHY lab). `must_be_same_term=true` means lecture + lab must appear together within the same term in `courses_by_term`.

### Example 3 - Bucket slot (12 credits of liberal arts)

Slot label: "Liberal Arts Electives". `slot_rule` filters completed courses by `department_ids_any` and optional course-number bounds, then sums credits and compares to `min_credits_required`.

## Design intent and why this scales

- Start small and grow: add more courses and requirements without rewriting the audit engine.
- Hard edge cases (test scores, permissions) live in `prereq_notes` for V1 so you can ship.
- Term-structured student data enables co-requisite constraints like "same term lecture + lab".

Last note

This document explains the intent behind the UML and how UniBrain should interpret each relationship. As UniBrain evolves, you can extend `slot_rule` keys, add more bundle constraints, or introduce track-level groups without breaking the core structure.